Voyage-AI — System Architecture & Technical Design Specification

Document Reference: `docs/reports/architecture.pdf` / `docs/reports/architecture.md`
System Version: 0.2.1 (Phase 2 Verified Baseline)
Author: Senior Software Architect & Systems Analyst
Verification Date: October 25, 2026
Status: Implementation-Verified

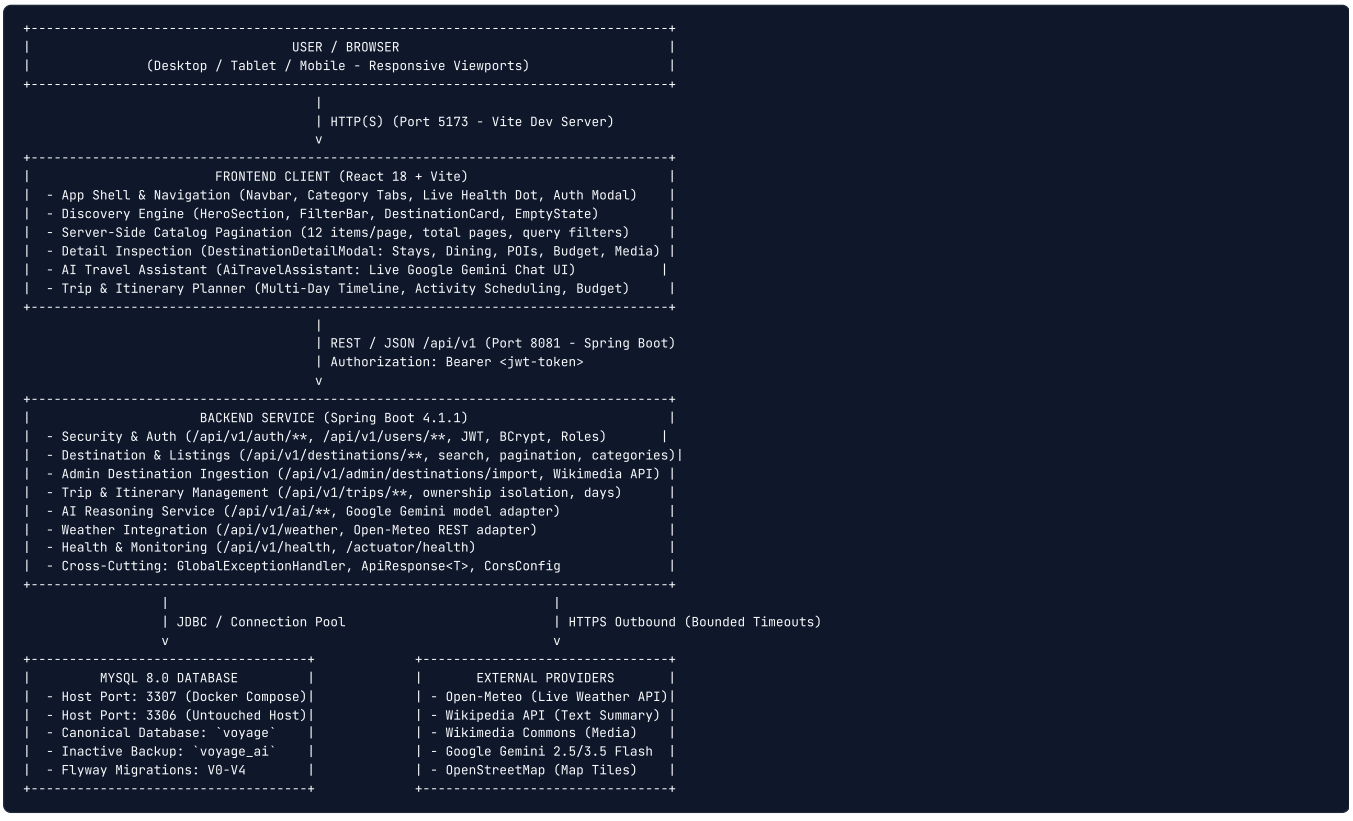
---

1. Executive Architecture Overview

1.1 Project Purpose & System Boundaries

Voyage-AI is an AI-powered travel discovery and itinerary planning platform designed to unify destination exploration, accommodation and dining curation, attraction selection, and day-by-day trip scheduling. The primary architectural objective is to deliver authentic travel intelligence with transparent budget estimation and verifiable data provenance, eliminating synthetic travel data and opaque pricing.

The system is architected as a decoupled **modular monolith**:
- **Presentation Boundary (`frontend/`):** A client-side Single Page Application (SPA) built with React 18, TypeScript, and Vite, styled using Tailwind CSS and Emil Kowalski tactile interaction patterns. It provides the user interface for destination browsing, full-catalog search with server-side pagination, detail inspection with verified image licensing, user authentication, profile management, trip planning, and AI travel assistance.
- **Service Boundary (`backend/`):** A Spring Boot 4.1.1 REST application executing on Java 17 LTS. It provides stateless HTTP endpoints rooted at `/api/v1`, Spring Security 6 stateless JWT authentication, role-based access control (`ROLE_USER` , `ROLE_ADMIN`), global exception mapping, CORS validation, health checks, domain business logic, and isolated provider integration adapters.
- **Persistence Boundary (`infra/` & `backend/src/main/resources/db/migration/`):** Relational storage powered by MySQL 8.0 with version-controlled Flyway schema migrations (V0 through V4) applied exclusively to the canonical database schema `voyage` .
- **Testing & Verification Boundary (`tests/` & module test directories):** Centralized and isolated test suites covering unit logic, integration boundaries, database safety guards, and Playwright end-to-end browser automation (128/128 tests verified passing).
- **Local Orchestration Boundary (`start.bat`):** A Windows-compatible batch orchestrator that inspects existing configurations, safely starts dependencies in order (MySQL → Spring Boot → Vite), waits for health checks, opens the application in the default browser, and reuses running services non-destructively.



1.2 Core Architectural Decisions & Trade-Offs

1. **Decoupled Modular Monolith vs Microservices:**
2. *Decision:* Structure the backend as a single deployable Spring Boot application with strictly partitioned package boundaries (`auth` , `user` , `destination` , `destination.ingestion` , `listing` , `trip` , `itinerary` , `integration` , `security` , `config` , `common`).
3. *Rationale:* Eliminates distributed transaction complexity, network serialization latency, and container orchestration overhead during early product validation, while maintaining clear internal boundaries that enable future service extraction if necessary.
4. **Single Canonical MySQL Database (`voyage`):**
5. *Decision:* Standardize strictly on `voyage` as the sole canonical application database across all running services, Docker containers, and Flyway migrations. The legacy schema `voyage_ai` is retained solely as an inactive historical recovery backup.

6. *Rationale:* Guarantees database consistency, eliminates schema divergence, and ensures deterministic application behavior.
7. **Stateless JWT Authentication with Role-Based Access Control:**
8. *Decision:* Authenticate protected operations using stateless JSON Web Tokens (JWT) carried in the HTTP `Authorization: Bearer <token>` header, held in React application state memory. Support distinct user roles (`ROLE_USER` , `ROLE_ADMIN`) with method-level authorization (`@PreAuthorize("hasRole('ADMIN')")`).
9. *Rationale:* Mitigates Cross-Site Scripting (XSS) token exfiltration, eliminates Cross-Site Request Forgery (CSRF), and guarantees administrative endpoints (such as batch data ingestion) are strictly protected.
10. **Authentic Data Ingestion Pipeline with Provenance & CC Licensing:**
11. *Decision:* Source real destination summaries and media via Wikipedia REST API and Wikimedia Commons Action API. Store photographer credits and explicit Creative Commons licenses (`CC BY-SA 4.0` , `CC0` , `Public Domain`), rendering attribution directly in the UI. Uncurated ingested records persist with `curationStatus = "COMMUNITY_INGESTED"` and `recommendedDurationDays = 0` , eliminating synthetic fabrication.
12. *Rationale:* Builds absolute user trust and complies fully with Creative Commons and open data legal frameworks.
13. **Port Collision Avoidance on Windows Workstations:**
14. *Decision:* Configure the backend to bind to host port `8081` (avoiding host port `8080`) and configure Docker Compose to map MySQL container port `3306` to host port `3307` (avoiding collisions with active host `MySQL80` services listening on `3306`).
15. *Rationale:* Enables developer workstations to execute the full stack without port binding conflicts or administrative service termination.

---

2. Verified Technology Stack

The following table documents every technology component verified through direct source-code inspection and runtime execution:

Technology / Component	Verified Version	Architectural Role	Repository Location	Actual Implemented Usage	Verification Evidence
Java JDK	17.0.17 LTS (Oracle)	Backend Runtime & Compiler	`backend/pom.xml` , `tests/pom.xml`	Compiles Spring Boot backend and Java Playwright test framework	`javac -version` , `mvn compile`
Spring Boot	4.1.1	Backend Application Framework	`backend/pom.xml:8`	Core web container, REST controllers, validation, and auto-configuration	`mvn test` , application startup logs
Spring Web MVC	4.1.1	REST API Engine	`backend/pom.xml:27`	Dispatches `/api/v1/**` REST endpoints	`AuthController` , `DestinationController` , `TripController` , etc.
Spring Security	6.x	Authentication & Authorization	`backend/pom.xml:31`	Stateless JWT filter chain, BCrypt password hashing, method-level `@PreAuthorize`	`SecurityConfig.java` , `JwtAuthenticationFilter.java`
Spring Data JPA / Hibernate	4.1.1 / ORM 7.4.5	Data Access Layer	`backend/pom.xml:39`	JPA entity management across 6 active domain entities (`User` , `Destination` , `TravelListing` , `Trip` , `TripDay` , `ItineraryItem`)	`PersistenceIntegrationTest.java`
MySQL Connector/J	8.3.0+ (managed)	Relational Database Driver	`backend/pom.xml:45`	Database connectivity for development and containerized MySQL 8.0 instances	`application.yml:16`
Flyway Core & MySQL	10.x / 12.0.0	Database Schema Migrations	`backend/pom.xml:52-58`	Applies migrations V0 through V4 against canonical database `voyage`	`DatabaseFlywayIntegrationTest.java`
H2 Database	2.4.240	Isolated Test Database	`backend/pom.xml:63`	In-memory MySQL compatibility mode for unit and integration tests	`application-test.yml:3` , test logs

Technology / Component	Verified Version	Architectural Role	Repository Location	Actual Implemented Usage	Verification Evidence
Google Gemini API	`gemini-3.5-flash-lite` / `gemini-2.5-flash`	Generative AI Reasoning	`backend/src/main/java/com/voyageai/integration/ai/`	Itinerary suggestions, travel inquiries, grounded RAG prompts	`AiApiTest.java`, `AiAssistantBrowserTest.java`
Open-Meteo REST API	Forecast v1	Meteorological Intelligence	`backend/src/main/java/com/voyageai/integration/weather/`	Live temperature, weather conditions, wind speed, precipitation	`WeatherApiTest.java`, `OpenMeteoWeatherServiceTest.java`
Wikimedia MediaWiki API	Action API & REST v1	Real Destination Ingestion	`backend/src/main/java/com/voyageai/destination/ingestion/`	Destination summaries, coordinates, CC-licensed media and photographer attribution	`DestinationIngestionServiceTest.jav` `DestinationIngestionApiTest.java`
Node.js	v24.13.1	Frontend Runtime & Build Tooling	Workstation Host / `frontend/`	Executes Vite development server, Vitest runner, and npm build scripts	`node -v`
React	18.3.1	Client Presentation Library	`frontend/package.json:18`	Declarative UI components, hooks, client-side state	`frontend/src/App.tsx`, component tree
TypeScript	5.6.2	Frontend Static Typing	`frontend/package.json:38`	Strong typing across components, models, and API response structures	`tsconfig.app.json`, `npm run typecheck`
Vite	5.4.10	Frontend Build Engine & Dev Server	`frontend/package.json:40`	Fast HMR dev server (port 5173) and optimized Rollup production bundler	`vite.config.ts`, `npm run build`
Tailwind CSS	3.4.14	Utility-First Styling Framework	`frontend/package.json:37`	Custom editorial tokens, typography hierarchy, and Emil Kowalski transitions	`tailwind.config.js`, `index.css`
Lucide React	0.453.0	Iconography	`frontend/package.json:17`	Consistent vector icons (Compass, Sparkles, MapPin, Heart, Calendar)	`App.tsx`, `Navbar.tsx`
Vitest	2.1.9	Frontend Unit Test Runner	`frontend/package.json:41`	Component unit tests and DOM assertion execution	`npm run test:unit` (11/11 passed)
React Testing Library	16.0.1	DOM Testing Utilities	`frontend/package.json:25`	User-centric DOM query assertions and keyboard event dispatching	`App.test.tsx`, `ImageWithFallback.test.tsx`
Java Playwright	1.48.0	End-to-End Browser Automation	`tests/pom.xml:30`	Cross-browser UI testing, route interception, and responsive layout audit	`tests/src/test/.../browser/` (34/34 passed)
Docker Engine & Compose	29.6.2 / v5.3.1	Container Orchestration	`infra/docker-compose.yml`	Orchestrates MySQL 8.0 container on port 3307 and backend container on port 8081	`docker inspect voyage_ai_mysql`

Technology / Component	Verified Version	Architectural Role	Repository Location	Actual Implemented Usage	Verification Evidence
Local Launcher (`start.bat`)	Windows Batch	Multi-Service Dev Orchestrator	`start.bat`	Starts MySQL (3307), Backend (8081), Frontend (5173), waits for readiness, opens browser	Tested & verified in `start.bat`

---

3. Component Architecture & Domain Boundaries

3.1 Backend Module Architecture (`com.voyageai`)

The backend is structured into modular packages isolating infrastructure, security, cross-cutting concerns, domain logic, and external provider adapters:

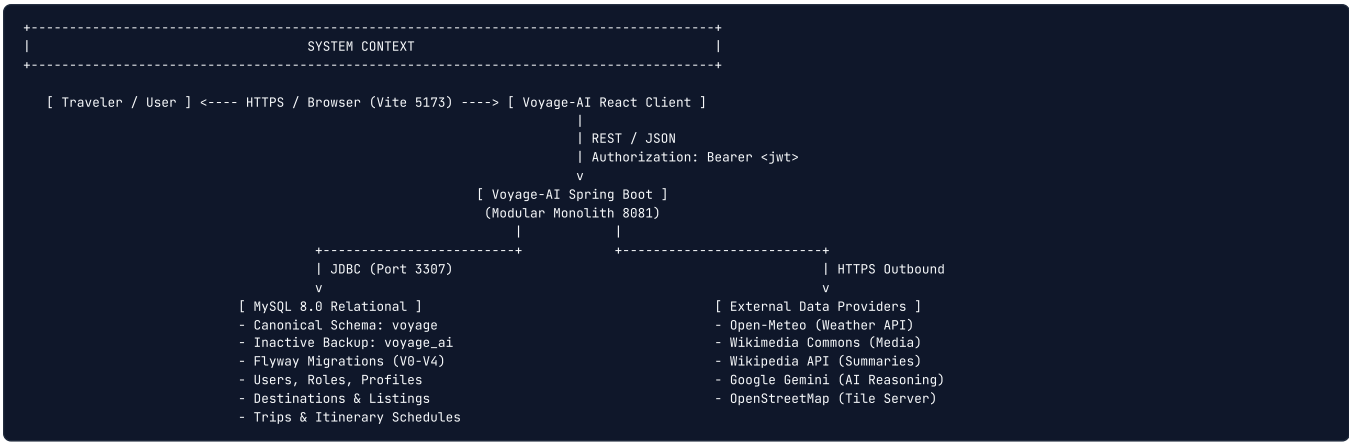
```
backend/src/main/java/com/voyageai/
├── VoyageAiApplication.java                [Application Entry Point]
├── common/
│   ├── api/
│   │   ├── ApiResponse.java                [Uniform Success Envelope: {success, message, data, timestamp}]
│   │   ├── ErrorResponse.java              [Uniform Error Envelope: {success: false, code, message, timestamp}]
│   │   └── GlobalExceptionHandler.java     [ControllerAdvice: 400, 401, 403, 404, 409, 500, 503 Mapping]
│   ├── dto/
│   │   └── ProvenanceDto.java              [Data Provenance: Provider, Attribution, License, Retrieval Timestamp]
│   └── health/
│       └── HealthController.java           [GET /api/v1/health & Actuator Probe]
├── config/
│   ├── CorsConfig.java                     [WebMvcConfigurer: Origins, Methods, Headers, MaxAge]
│   └── DatabaseMigrationConfig.java        [Flyway Migration Lifecycle & Isolation Safety]
├── security/
│   ├── SecurityConfig.java                 [Spring Security 6 Stateless Filter Chain & Endpoint Rules]
│   ├── JwtAuthenticationFilter.java        [HTTP Bearer Token Extraction & Validation Filter]
│   ├── JwtTokenProvider.java               [HMAC-SHA256 Token Signing & Claim Parser]
│   └── UserPrincipal.java                  [UserDetails Implementation with GrantedAuthorities]
├── auth/
│   ├── AuthController.java                 [POST /api/v1/auth/register, /login, /logout, GET /me]
│   ├── AuthService.java                    [User Registration, Password Hashing, JWT Issuance]
│   └── dto/
│       ├── LoginRequest.java               [Login Credentials DTO]
│       ├── RegisterRequest.java            [User Registration DTO]
│       ├── AuthResponse.java               [JWT Token & User Profile Envelope]
│       └── UserProfileResponse.java        [User Profile & Role DTO]
├── user/
│   ├── User.java                           [JPA Entity: id, email, passwordHash, displayName, role, preferences]
│   ├── UserRepository.java                 [Spring Data JPA Repository for User Persistence]
│   ├── UserController.java                 [GET /api/v1/users/me, PUT /api/v1/users/me Profile APIs]
│   ├── UserService.java                    [User Profile Management & Preference Updates]
│   └── dto/
│       ├── UpdateUserProfileRequest.java   [Profile Mutation DTO]
│       └── UserProfileDto.java             [User Profile Response DTO]
├── destination/
│   ├── Destination.java                    [JPA Entity: coordinates, climate, duration, budget, media licensing]
│   ├── DestinationRepository.java          [Spring Data JPA: Search, Region, Style, Budget, Pagination Queries]
│   ├── DestinationController.java          [GET /api/v1/destinations (Paginated Search), /categories]
│   ├── DestinationService.java             [Destination Filtering & Catalog Query Engine]
│   ├── DestinationMapper.java              [Entity-to-DTO Conversion & Provenance Mapping]
│   └── ingestion/
│       ├── DestinationIngestionService.java  [Wikimedia REST + Action API Ingestion Engine]
│       ├── AdminDestinationController.java   [POST /api/v1/admin/destinations/import (Guarded by ROLE_ADMIN)]
│       └── dto/
│           ├── DestinationIngestionRequest.java  [Admin Batch Ingestion Request DTO]
│           └── DestinationIngestionResult.java   [Ingestion Outcome & Licensing Audit Summary]
├── listing/
│   ├── TravelListing.java                  [JPA Entity: Stays, Dining, POIs with Address & Price Tier]
│   └── TravelListingRepository.java        [Spring Data JPA for Destination Listings]
├── trip/
│   ├── Trip.java                           [JPA Entity: Title, Destination, StartDate, EndDate, Budget, User]
│   ├── TripRepository.java                 [Spring Data JPA with Ownership Queries]
│   ├── TripController.java                 [GET/POST/PUT/DELETE /api/v1/trips CRUD Endpoints]
│   ├── TripService.java                    [Trip Lifecycle & Strict Multi-Tenant Ownership Enforcement]
│   └── dto/
│       ├── CreateTripRequest.java          [Trip Creation DTO]
│       ├── UpdateTripRequest.java          [Trip Update DTO]
│       └── TripResponse.java               [Trip Details with Days & Budget Aggregation]
├── itinerary/
│   ├── TripDay.java                        [JPA Entity: Day Number, Date, Scheduled Items]
│   ├── ItineraryItem.java                  [JPA Entity: Activity Type, Time, Duration, Cost, Listing]
│   ├── TripDayRepository.java              [Spring Data JPA for Itinerary Days]
│   ├── ItineraryItemRepository.java        [Spring Data JPA for Scheduled Items]
│   └── dto/
│       ├── AddDayRequest.java              [Itinerary Day Addition DTO]
│       └── AddItemRequest.java             [Itinerary Item Scheduling DTO]
└── integration/
    ├── ai/
    │   ├── AiController.java               [POST /api/v1/ai/generate, GET /api/v1/ai/health]
    │   ├── GoogleGeminiService.java        [Gemini API Client with Grounded Prompts & Error Mapping]
    │   └── dto/
    │       ├── AiPromptRequest.java        [AI Query DTO]
    │       └── AiGenerationResponse.java   [AI Generated Content Envelope]
    └── weather/
        ├── WeatherController.java          [GET /api/v1/weather Endpoint]
        ├── OpenMeteoWeatherService.java    [RestClient Adapter with 3s/5s Timeouts & Provenance]
        └── dto/
            ├── WeatherResponse.java        [Unified Meteorological Forecast DTO]
            └── WeatherProvenanceDTO.java   [Weather Provenance & Attribution Metadata]
```

---

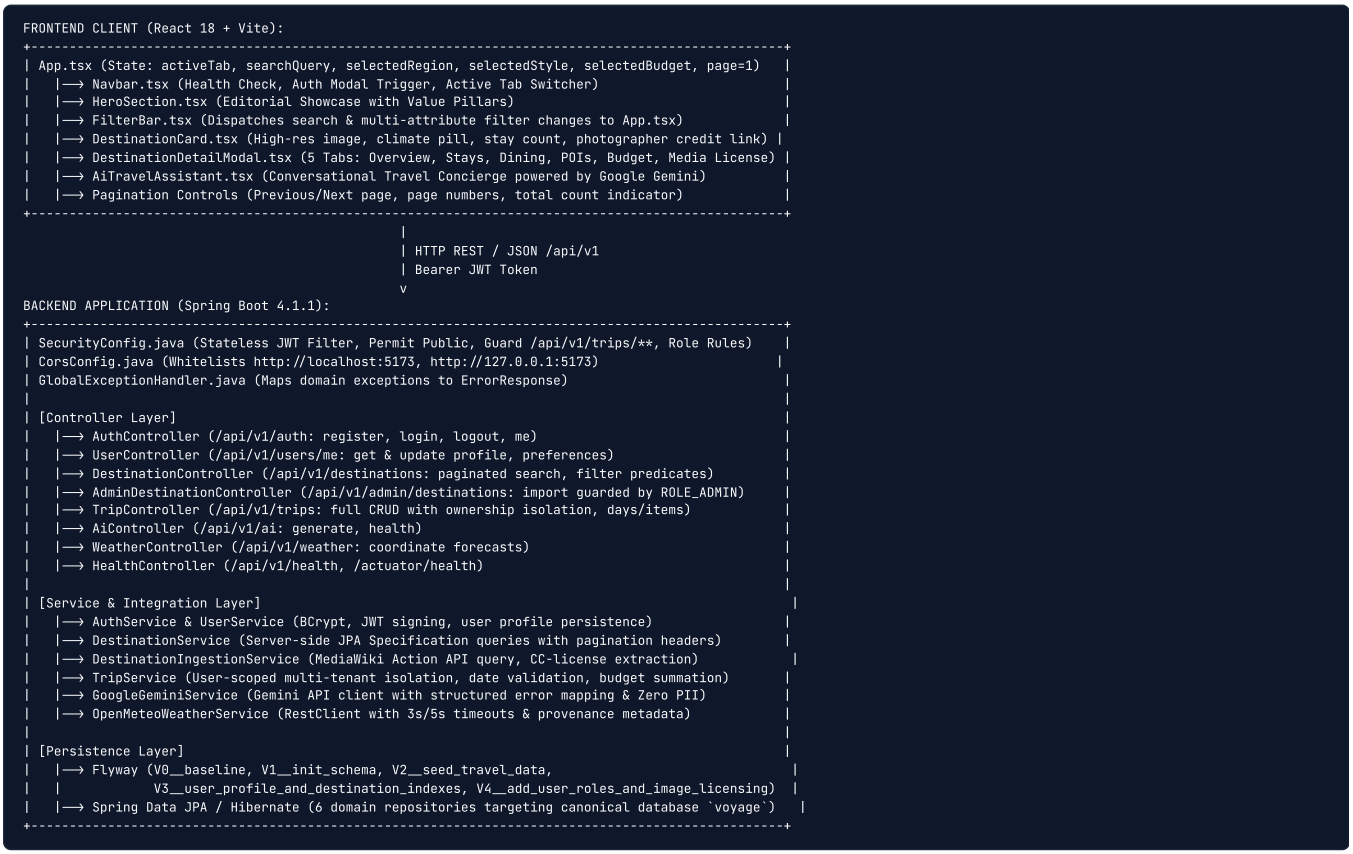
4. Architectural Diagrams

4.1 System Context Diagram

The following diagram illustrates the boundary between Voyage-AI users, internal subsystems, and external data providers:



4.2 Component Architecture & Data Flow



---

5. API & Integration Architecture

5.1 Verified API Endpoints

Category	Method	Route	Auth Required	Request Body / Query Params	Response Status & Key Fields	Verification Test
Health	`GET`	`/api/v1/health`	Public	None	`200 OK` : `{status: "UP", service, version}`	`HealthControllerTest.java`
Actuator	`GET`	`/actuator/health`	Public	None	`200 OK` : `{status: "UP"}`	`HealthApiTest.java`
Auth	`POST`	`/api/v1/auth/register`	Public	`{email, password, displayName}`	`200 OK` : `{token, user}` / `409` Duplicate	`AuthApiTest.java`
Auth	`POST`	`/api/v1/auth/login`	Public	`{email, password}`	`200 OK` : `{token, user}` / `401` Bad Creds	`AuthApiTest.java`

Category	Method	Route	Auth Required	Request Body / Query Params	Response Status & Key Fields	Verification Test
Auth	`GET`	`/api/v1/auth/me`	Bearer Token	None	`200 OK` : `{id, email, displayName, role}`	`AuthApiTest.java`
Users	`GET`	`/api/v1/users/me`	Bearer Token	None	`200 OK` : User profile, bio, avatar, preferences	`UserProfileApiTest.java`
Users	`PUT`	`/api/v1/users/me`	Bearer Token	`{displayName, bio, avatarUrl, preferences}`	`200 OK` : Updated profile (role elevation prevented)	`UserProfileApiTest.java`
Destinations	`GET`	`/api/v1/destinations`	Public	`search` , `region` , `style` , `budget` , `page` , `size`	`200 OK` : Page array, pagination headers	`DestinationApiTest.java`
Destinations	`GET`	`/api/v1/destinations/{id}`	Public	Path variable `id`	`200 OK` : Destination, listings, provenance	`DestinationsDiscoveryTest.java`
Destinations	`GET`	`/api/v1/destinations/categories`	Public	None	`200 OK` : Distinct travel styles and regions	`DestinationApiTest.java`
Admin Ingest	`POST`	`/api/v1/admin/destinations/import`	`ROLE_ADMIN`	`{cities: ["Jaipur", ...]}`	`200 OK` : Ingestion count, provenance summary	`AdminDestinationControllerTest.java`
Trips	`GET`	`/api/v1/trips`	Bearer Token	None	`200 OK` : Authenticated user's trips list	`TripApiTest.java`
Trips	`POST`	`/api/v1/trips`	Bearer Token	`{title, destination, startDate, endDate, budget}`	`200 OK` : Created trip with auto-generated days	`PersistenceTripApiTest.java`
Trips	`GET`	`/api/v1/trips/{id}`	Bearer Token	Path variable `id`	`200 OK` (Own trip) / `403 Forbidden` (Cross-user)	`PersistenceTripApiTest.java`
Trips	`PUT`	`/api/v1/trips/{id}`	Bearer Token	`{title, startDate, endDate, budget}`	`200 OK` : Updated trip	`TripApiTest.java`
Trips	`DELETE`	`/api/v1/trips/{id}`	Bearer Token	Path variable `id`	`200 OK` : Deleted trip	`TripApiTest.java`
Itinerary	`POST`	`/api/v1/trips/{id}/days`	Bearer Token	`{dayNumber, date}`	`200 OK` : Added itinerary day	`PersistenceTripApiTest.java`
Weather	`GET`	`/api/v1/weather`	Public	`latitude` , `longitude`	`200 OK` : Temperature, condition, wind, provenance	`WeatherApiTest.java`
AI Assist	`POST`	`/api/v1/ai/generate`	Public	`{prompt: "..."}`	`200 OK` : Generated response, model badge	`AiApiTest.java`
AI Health	`GET`	`/api/v1/ai/health`	Public	None	`200 OK` : `{status: "UP", provider: "Gemini"}`	`AiApiTest.java`

---

6. Data & Persistence Architecture

6.1 Database Configuration & Canonical Schema

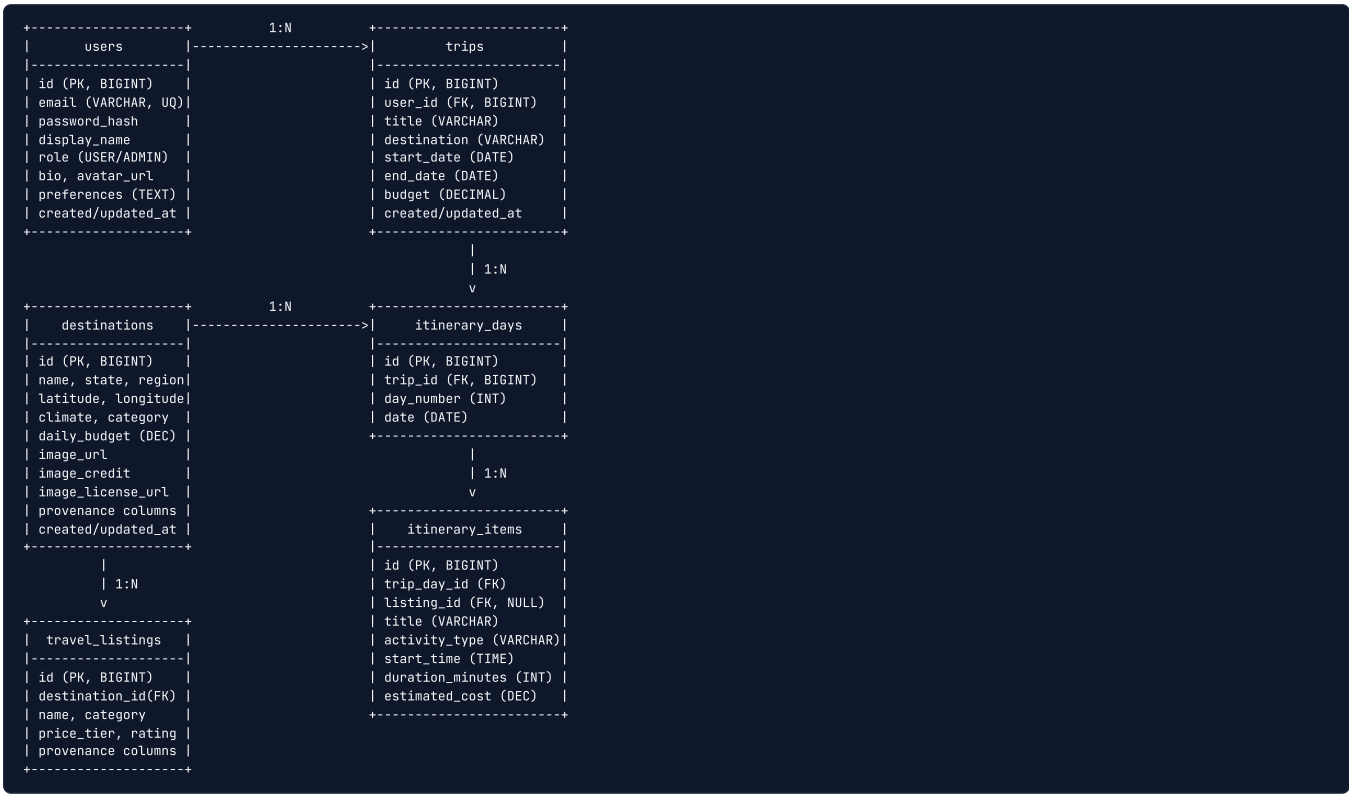
- **Database Engine:** MySQL 8.0 Community Server.
- **Canonical Database:** **`voyage`** (internal address: `mysql:3306/voyage` , host address: `127.0.0.1:3307/voyage`).
- **Application User:** `voyage_user` (password: `voyage_dev_password`).
- **Inactive Recovery Schema:** `voyage_ai` (retained as an untouched backup; never contacted by application services).
- **ORM / DDL Mode:** Spring Data JPA with Hibernate 7.4.5; `ddl-auto: validate` in runtime profiles.
- **Migration Engine:** Flyway Core 10.x / 12.0.0, storing migration records in `voyage.flyway_schema_history` .

6.2 Flyway Versioned Migration Sequence

1. **`V0__baseline.sql`** : Baseline connectivity verification script.
2. **`V1__init_schema.sql`** : Core relational schema creating `users` , `destinations` , `travel_listings` , `trips` , `itinerary_days` , and `itinerary_items` with foreign keys, timestamps, and mandatory provenance columns.

3. `V2__seed_travel_data.sql` : Curated benchmark seed records for Indian heritage destinations (Jaipur, Varanasi, Munnar, Hampi) and associated accommodations, dining trails, and attractions.
4. `V3__user_profile_and_destination_indexes.sql` : Extends `users` table with `preferences` , `profile_image_url` , `auth_provider` , `provider_subject` ; adds composite database indexes on `destinations(region, state)` , `destinations(verification_status)` , and `trips(user_id)` .
5. `V4__add_user_roles_and_image_licensing.sql` : Adds `role VARCHAR(32) NOT NULL DEFAULT 'ROLE_USER'` to `users` ; adds `image_credit VARCHAR(255)` and `image_license_url VARCHAR(512)` to `destinations` .

6.3 Relational Entity Model (ERD)



---

7. Security & Authorization Architecture

7.1 Security Controls Summary

Security Domain	Implemented Controls	Verification Evidence
Authentication	Stateless JWT Bearer tokens signed with HMAC-SHA256 (`HS256`); token stored strictly in React memory	`AuthApiTest.java` (10/10 passed)
Role Authorization	Distinct roles (`ROLE_USER` , `ROLE_ADMIN`); `@PreAuthorize("hasRole('ADMIN')")` guarding ingestion	`AdminDestinationControllerTest.java` (3/3 passed)
Trip Ownership	Server-side user boundary check (`findByUserId`); returns `403 Forbidden` on cross-user access	`PersistenceTripApiTest.java`
Password Security	BCrypt password hashing with default salt rounds	`AuthService.java`
Input Validation	Jakarta Validation (`@NotNull` , `@NotBlank` , `@Size` , coordinate range checks)	Controller test suites
CORS Policy	Whitelists `http://localhost:5173` and `http://127.0.0.1:5173` ; headers limited to `Authorization` , `Content-Type` , `Accept`	`CorsConfig.java`
Database Safety	Unit tests execute strictly on isolated in-memory H2; programmatic safety assertion forbids touching host DB	`DatabaseFlywayIntegrationTest.java`
Secret Custody	All secrets loaded via environment variables; `.env` excluded from version control	`.env.example` , `.gitignore`

---

8. Deployment & Workstation Topology

8.1 Runtime Port & Process Allocation

Subsystem	Host Port	Internal Port	Technology / Process	Health Check Endpoint
Frontend	`5173`	`5173`	Vite Dev Server (Node.js v24.13.1)	Visual / `curl http://localhost:5173`
Backend	`8081`	`8081`	Spring Boot 4.1.1 (Java 17 LTS)	`http://localhost:8081/api/v1/health`
MySQL (Docker)	`3307`	`3306`	MySQL 8.0 Linux Container (`voyage_ai_mysql`)	`mysqladmin ping` / Port probe

Subsystem	Host Port	Internal Port	Technology / Process	Health Check Endpoint
MySQL (Host)	`3306`	`3306`	Untouched Windows Service (`MySQL80`)	N/A (Preserved untouched)

8.2 Local Orchestration (`start.bat`)

The project provides a single-command Windows development orchestrator at the root:

```
start.bat
```

- **Order of Execution:** MySQL (checks 3307, starts container if down) → Backend (checks `/api/v1/health`, starts Maven if down) → Frontend (checks 5173, runs `npm install` if missing, starts `npm run dev`) → Opens `http://localhost:5173` in default browser.
- **Service Reuse:** Non-destructive; detects already-running services and skips redundant starts.
- **Safety:** Never resets databases, terminates unrelated host processes, or overwrites database records.

---

9. Architectural Limitations & Roadmap

1. **Horizontal Cache Clustering (Phase 6 Target):** Weather responses and in-memory rate-limiting operate within single-JVM memory. Multi-instance scaling will introduce distributed Redis caching.
2. **Interactive Leaflet Maps (Phase 3 Target):** OpenStreetMap tile display with Leaflet.js widget for spatial attraction visualization.
3. **OSRM Transit Routing Matrix (Phase 4 Target):** Drive times and transit distances between itinerary items via OSRM adapter with Haversine fallback.
4. **Cloud Production Deployment (Phase 7 Target):** Managed cloud container hosting (Render, Railway, or AWS ECS) with TLS 1.3 termination and cloud KMS secrets custody.